

**LAPORAN PRAKTIKUM
SISTEM OPERASI**

TUGAS 3

Dibuat untuk Memenuhi Tugas Mata Kuliah Sistem Operasi



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.K

Disusun Oleh Kelompok 3:

Muhamad Gufron (2410651053)

**PROGRAM STUDI TEKNIK INFORMATIKA FAKULTAS
TEKNIK**

**UNIVERSITAS UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN
AKADEMIK 2024/2025**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada Triawan Adi Cahyanto, M.K selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini.

Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

PEMBAHASAN

1.1. Kalkulator_paralel.c

pada Kode C ini mengimplementasikan kalkulator yang menggunakan multithreading POSIX (pthread) untuk menjalankan empat operasi aritmatika (**tambah, kurang, kali, bagi**) secara **paralel**. Data input dan hasilnya disimpan dalam struktur Data. Program utama (**main thread**) membuat empat *thread* terpisah, masing-masing menghitung satu operasi. Fungsi `pthread_join()` digunakan untuk memastikan *main thread* menunggu hingga semua perhitungan selesai, menjamin hasil akurat sebelum menampilkannya kepada pengguna

```
gufro@MSI:~$ cd
gufro@MSI:~$ cd ~/Praktikum3
gufro@MSI:~/Praktikum3$ nano kalkulator_paralel.c
gufro@MSI:~/Praktikum3$ gcc kalkulator_paralel.c -o kalkulator_paralel
gufro@MSI:~/Praktikum3$ ./kalkulator_paralel
Masukkan dua angka: 10 5

=== Hasil Operasi ===
Penjumlahan: 15.00
Pengurangan: 5.00
Perkalian   : 50.00
Pembagian   : 2.00
gufro@MSI:~/Praktikum3$ |
```

`#include <stdio.h>` // gunakan untuk input dan output

`#include <pthread.h>` // digunakan untuk membuat dan mengatur thread

`/` //pada Variabel a dan b digunakan untuk menyimpan angka

`// pad Variabel hasil digunakan untuk menyimpan hasil operasi`

`typedef struct {`

`double a, b;`

`double hasil;`

`} Data;`

`// digunakan untuk satu thread`

`// arg berisi struktur data yang di casting pada data`

`// dan menghitung hasil A dan B yang di hitung menggunakan (+, -, *, /) lalu keluaran dengan pthread_exit(NULL)`

```

void* tambah(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a + data->b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a - data->b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a * data->b;
    pthread_exit(NULL);
}

void* bagi(void* arg) {
    Data* data = (Data*)arg;
    if (data->b != 0)
        data->hasil = data->a / data->b;
    else
        data->hasil = 0; // hindari pembagian nol
    pthread_exit(NULL);
}

int main() {
    pthread_t tTambah, tKurang, tKali, tBagi;
    Data dTambah, dKurang, dKali, dBagi;
    double x, y;

    // pada pthread ini digunakan untuk menyimpan tipe data ID thread. Dan membuat 4
    thread yang berbeda operasi

    printf("Masukkan dua angka: ");

```

```

scanf("%lf %lf", &x, &y);

    // pada di atas digunakan untuk memasukkan dua angka yang digunakan
    // Isi data untuk masing-masing operasi
dTambah.a = dKurang.a = dKali.a = dBagi.a = x;
dTambah.b = dKurang.b = dKali.b = dBagi.b = y;

    // pada 4 thread ini menjalankan bersamaan, masing-masing menghitung operasi
    sendiri

pthread_create(&tTambah, NULL, tambah, &dTambah);
pthread_create(&tKurang, NULL, kurang, &dKurang);
pthread_create(&tKali, NULL, kali, &dKali);
pthread_create(&tBagi, NULL, bagi, &dBagi);

// pada pthread digunakan untuk agar program utama menunggu setiap program selesai
    Jika sudah selesai lanjut berikutnya
pthread_join(tTambah, NULL);
pthread_join(tKurang, NULL);
pthread_join(tKali, NULL);
pthread_join(tBagi, NULL);

// pada printf digunakan untuk menampilkan hasil setiap operasi
printf("\n=== Hasil Operasi ===\n");
printf("Penjumlahan: %.2lf\n", dTambah.hasil);
printf("Pengurangan: %.2lf\n", dKurang.hasil);
printf("Perkalian : %.2lf\n", dKali.hasil);
printf("Pembagian : %.2lf\n", dBagi.hasil);

return 0; // menandakan operasi selesai
}

```

Berikut penjenjelasn proses komplikasi dan eksekusi pada berikut ini :

1. gufro@MSI:~\$ cd, gufro@MSI:~\$ cd ~/Praktikum3 dan nano kalkulator_parallel.c

- digunakan direktori kerja ~/Praktikum3
 - mengubah direktori pada terminal, yang penting karena file kode sumber harus diakses
 - nano di gunakan membuka text editor untuk menulis atau memodifikasi kode
2. gufro@MSI:~/Praktikum3\$ gcc kalkulator_parallel.c -o kalkulator_parallel
 - gcc kalkulator_parallel.c -o kalkulator_parallel digunakan untuk menyebutkan file kode sumber yang di komplikasikan dan memberi hasil nama file output yang di eksekusi pada biner
 - hasil: jika tidak ada kesalahan sintaks yang di perintahkan menghasilkan file biner baru yang siap di jalankan biasanya memerlukan penambah flag -pthread saat di komplikasi, flag ini tidak wajib.
 3. gufro@MSI:~/Praktikum3\$./kalkulator_parallel
 - di gunakan untuk membuat dan menjalankan file program biner yang telah di komplikasi
 - setelah menjalankan semuanya program telah selesai di eksekusi pada terminal dan menampilkan hasil output yang di jalankan

1.2. Nano file_processing.c

menunjukkan proses kompilasi dan eksekusi program C bernama file_processing.c di terminal Linux. Program ini, yang dikompilasi dengan *flag* -lpthread untuk mendukung multithreading, bertujuan menghitung jumlah kata dalam sebuah file. Hasil eksekusi membandingkan performa antara metode multithread dan single-thread. Meskipun kedua metode menghasilkan jumlah kata yang sama terlihat jelas bahwa untuk kasus ini, eksekusi single-thread jauh lebih dibandingkan multithread, menyiratkan *overhead* multithreading yang signifikan pada operasi I/O atau beban kerja ringan.

```
gufro@MSI:~/Praktikum3$ nano file_processing.c
gufro@MSI:~/Praktikum3$ gcc file_processing.c -o file_processing -lpthread
gufro@MSI:~/Praktikum3$ ./file_processing
Jumlah kata (multithread): 19
Waktu eksekusi multithread: 1.756 ms
Jumlah kata (single-thread): 19
Waktu eksekusi single-thread: 0.003 ms
gufro@MSI:~/Praktikum3$ nano input.txt
gufro@MSI:~/Praktikum3$ gcc file_processing.c -o file_processing -lpthread
gufro@MSI:~/Praktikum3$ ./file_processing
Jumlah kata (multithread): 19
Waktu eksekusi multithread: 0.731 ms
Jumlah kata (single-thread): 19
Waktu eksekusi single-thread: 0.001 ms
gufro@MSI:~/Praktikum3$ |
```

```
#include <stdio.h> // gunakan untuk input dan output
```

```
#include <stdlib.h> // fungsi umum(malloc,exit)
```

```

#include <pthread.h>

#include <string.h> // operasi string

#include <sys/time.h> // menghitung waktu


#define MAX_SIZE 1000000 // di gunakan batas maksimal ukuran file


// Struktur ini di gunakan untuk mengirim data pada setiap thread dan setiap thread
menghitung sebagian teks

typedef struct {
    char *text;

    int start; // posisi awal

    int end; // posisi akhir

    int count; // hasil jumlah kata
} ThreadData;


// Fungsi untuk menghitung jumlah kata pada bagian teks yang di berikan, dan setiap kali
karakter bukan spasi ditemukan setelah karakter spasi

void *countWords(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int count = 0;

    int inWord = 0;

    for (int i = data->start; i < data->end; i++) {
        if (data->text[i] == ' ' || data->text[i] == '\n' || data->text[i] == '\t') {
            inWord = 0;
        } else if (!inWord) {
            inWord = 1;
            count++;
        }
    }

    data->count = count;

    pthread_exit(NULL);
}

```

```
}
```

```
int countWordsSingle(char *text, int length) {  
    int count = 0;  
    int inWord = 0;  
    for (int i = 0; i < length; i++) {  
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t') {  
            inWord = 0;  
        } else if (!inWord) {  
            inWord = 1;  
            count++;  
        }  
    }  
    return count;  
}
```

// Fungsi ini sama dengan di atas hanya saja menghitung seluruh isi file tanpa di bagi dua dan tidak secara paralel

```
int main() {  
    FILE *file = fopen("input.txt", "r");  
    if (!file) {  
        printf("Gagal membuka file.\n");  
        return 1;  
    }  
}
```

```
char *text = malloc(MAX_SIZE);  
int length = fread(text, 1, MAX_SIZE, file);  
fclose(file);
```

// pada di atas digunakan membuka file input.txt untuk membaca seluruh isi yang berada pada variabel dan menutup file

// --- Versi Multithread ---


```

        // digunakan menghitung jumlah kata yang single- thread
struct timeval start, end;
gettimeofday(&start, NULL);

pthread_t t1, t2;
ThreadData data1 = {text, 0, length / 2, 0};
ThreadData data2 = {text, length / 2, length, 0};

pthread_create(&t1, NULL, countWords, &data1);
pthread_create(&t2, NULL, countWords, &data2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

int totalMulti = data1.count + data2.count;

gettimeofday(&end, NULL);
double timeMulti = (end.tv_sec - start.tv_sec) * 1000.0;
timeMulti += (end.tv_usec - start.tv_usec) / 1000.0;
        // pada di atas digunakan untuk membagi file 2 bagian awal – akhir
        //dan menjalankan untuk menghitung bagian masing masing. Meunggu kedua nya
        selesai pada thread di gabung data1 sampai data2

// --- Versi Single Thread ---
gettimeofday(&start, NULL);
int totalSingle = countWordsSingle(text, length);
gettimeofday(&end, NULL);
double timeSingle = (end.tv_sec - start.tv_sec) * 1000.0;
timeSingle += (end.tv_usec - start.tv_usec) / 1000.0;
        // digunakan untuk menguhitung kata menggunakan satu fungsi tanpa thread dan
        mengukur waktu

```

```

// --- Hasil ---

printf("Jumlah kata (multithread): %d\n", totalMulti);
printf("Waktu eksekusi multithread: %.3f ms\n", timeMulti);
printf("Jumlah kata (single-thread): %d\n", totalSingle);
printf("Waktu eksekusi single-thread: %.3f ms\n", timeSingle);

    // digunakan menampilkan jumlah kata dari kedua metode pada eksekusi

free(text);

return 0;

}

// digunakan untuk teks sebelum program selesai

```

Berikut penjelasan proses kompilasi dan eksekusi pada berikut ini:

1. nano file_processing.c
 - di gunakan untuk membuka editor teks nano dan menulis kode program c atau memodifikasi code
2. gcc file_processing.c -o file_processing -lpthread
 - gcc compile bahasa c
 - file_processing.c sumber kode program yang di buat
 - -o file_processing komplikasi dak di eksekusi pada file
 - -lpthread menautkan link pthread yang fungsinya multithreading yang hasilnya terbentuk biner
3. ./file_processing
 - Tanda ./ digunakan untuk menjalankan program
 1. Membuka file input.txt
 2. Membaca teks di dalam file
 3. Menghitung jumlah kata menggunakan 2 metode bekerja berurutan dan paralel
 4. Dan menampilkan hasil jumlahkata dan waktu di eksekusi
4. Meihat hasil eksekusi
 - Jumlah kata harus kedua metode benar dan hasil hitungnya benar tidak salah
 - Pada hasil di eksekusi berbeda karena File kecil single -thread lebih cepat sedangkan File besar multithreading bisa lebih efisien
5. nano input.txt
 - Digunakan untuk membuka file teks yang menjadi input data program

- Ketika file di ubah misalnya tambah teks program komplikasi ulang dan dijalankan maka dilihat perubahan berhasil

1.3. Produser_consumer.c

Program ini dijalankan dengan thread Producer bertugas menghasilkan data (angka) dan thread Consumer bertugas memprosesnya. Kedua proses ini berbagi buffer terbatas. Output memperlihatkan interaksi dinamis di mana Consumer mungkin menunggu saat buffer kosong, dan kemudian secara bergantian, Producer mengisi buffer dan Consumer mengosongkannya, menjaga thread tetap sinkron untuk mencegah race condition

```
gufro@MSI:~/Praktikum3$ nano producer_consumer.c
gufro@MSI:~/Praktikum3$ gcc producer_consumer.c -o producer_consumer
gufro@MSI:~/Praktikum3$ ./producer_consumer
Buffer kosong! Consumer menunggu...
Producer menghasilkan: 83 (isi buffer: 1)
Consumer memproses: 83 (isi buffer: 0)
Producer menghasilkan: 86 (isi buffer: 1)
Consumer memproses: 86 (isi buffer: 0)
Producer menghasilkan: 77 (isi buffer: 1)
Producer menghasilkan: 15 (isi buffer: 2)
Consumer memproses: 77 (isi buffer: 1)
Producer menghasilkan: 93 (isi buffer: 2)
Producer menghasilkan: 35 (isi buffer: 3)
Consumer memproses: 15 (isi buffer: 2)
Producer menghasilkan: 86 (isi buffer: 3)
Producer menghasilkan: 92 (isi buffer: 4)
Consumer memproses: 93 (isi buffer: 3)
Producer menghasilkan: 49 (isi buffer: 4)
Producer menghasilkan: 21 (isi buffer: 5)
Consumer memproses: 35 (isi buffer: 4)
Producer menghasilkan: 62 (isi buffer: 5)
Producer menghasilkan: 27 (isi buffer: 6)
Consumer memproses: 86 (isi buffer: 5)
```

```
#include <stdio.h> // digunakan untuk input dan output
```

```
#include <stdlib.h> // fungsi umum(malloc,exit)
```

```
#include <pthread.h> // digunakan untuk thread, mutex
```

```
#include <unistd.h> fungsi slepp()
```

```
#define BUFFER_SIZE 10
```

```

int buffer[BUFFER_SIZE];

int count = 0; // jumlah item dalam buffer

int in = 0; // posisi producer menulis

int out = 0; // posisi consumer membaca


pthread_mutex_t mutex; // digunakan mengunci akses buffer agar tidak di akses bersamaan

pthread_cond_t not_full; // digunakan untuk memberi sinyal ke consumer jika buffer tidak
tidak penuh

pthread_cond_t not_empty; // // digunakan untuk memberi sinyal ke consumer jika buffer
tidak tidak kosong


// Fungsi untuk menghasilkan angka random dan menambah ke buffer yang terus berjalan
dalam looping tak hingga, menghasilkan angka acak 0- 99
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // angka random 0-99

        pthread_mutex_lock(&mutex);

        // digunakan mengunci buffer agar tidak di akses thread sat di tulis

        // Tunggu jika buffer penuh jika buffer sudah penuh berisi 10 item maka producer tidak
        Tidak bisa menambah data baru

        while (count == BUFFER_SIZE) {

            printf("Buffer penuh! Producer menunggu...\n");

            pthread_cond_wait(&not_full, &mutex);

        }

        // Tambahkan item ke buffer

        buffer[in] = item; // item di simpan menggunakan in

        in = (in + 1) % BUFFER_SIZE; // jika in di naikkan kebalikan ke 0 jika mencapai akhir

        count++; //di gunakan menambah 1 item baru
    }
}

```

```

printf("Producer menghasilkan: %d (isi buffer: %d)\n", item, count);

// Beri sinyal ke consumer
pthread_cond_signal(&not_empty); // memberi tahu consumer tidak kosong lagi
pthread_mutex_unlock(&mutex); // membuka consumer agar dapat di akses ke buffer

sleep(1); // jeda untuk simulasi secara bergantian
}
return NULL;
}

// Fungsi untuk mengambil angka dari buffer dan memprosesnya
void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Tunggu jika buffer kosong consumer tidak mempunyai data untuk sdi proses
        while (count == 0) {
            printf("Buffer kosong! Consumer menunggu...\n");
            pthread_cond_wait(&not_empty, &mutex);
        }

        // Ambil item dari buffer
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        // consumer mengambil item dari posisi out bergeser satu langkah maju dan berputar
        // mencapai akhir count di kurangi 1 karena satu item sudah di ambil
        printf("Consumer memproses: %d (isi buffer: %d)\n", item, count);

        // memberi tahu ke producer bahwa masih ada ruang kosong

```

```

pthread_cond_signal(&not_full);
pthread_mutex_unlock(&mutex); // melepas produser bisa menambah lagi

sleep(2); // jeda untuk simulasi sedikit lebih lambat dari produser
}
return NULL;
}

int main() {
    pthread_t prod, cons;

    // Inisialisasi mutex dan kondisi agar siap digunakan.
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // Buat thread producer dan consumer menghasilkan angka acak dan mengambil dan
    memproses angka
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Tunggu thread selesai (tidak akan selesai karena loop infinite) karena program baris tidak
    akan pernah tercapai
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Membersihkan sumber daya setelah selesai (tidak dijalankan karena loop infinite).
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);
    return 0;
}

```

}

Berikut penjelasan proses komplikasi dan eksekusi pada program berikut ini :

1. nano producer_consumer.c
 - di gunakan untuk membuka editor teks nano dan menulis kode program atau memodifikasi code yang berisi logika consumer dengan thread, mutex, buffer

Komplikasi program

2. gcc producer_consumer.c -o producer_consumer
 - gcc digunakan untuk mengompilasi kode C.
 - produser_consumer.c nama file sumber (source code).
 - -o produser_consumer hasil kompilasi disimpan dalam file executable bernama produser_consumer
 - Saat proses ini, compiler akan melakukan:
 - Preprocessing menggabungkan header
 - Compilation mengubah kode C menjadi assembly.
 - Assembly mengubah assembly menjadi *object code* (.o).
 - Linking menggabungkan object code dengan library pthread untuk membentuk program jadi (executable file).
3. Eksekusi program
 - ./producer_consumer
Tanda ./ menunjukkan bahwa file executable berada di direktori yang sama dengan terminal.
4. Proses yang Terjadi Saat Program Dijalankan
 - Thread Producer berjalan menghasilkan angka acak (0–99)
 - Thread Consumer berjalan mengambil angka dari buffer dan memprosesnya.
 - Mutex dan Condition digunakan agar kedua thread tidak mengakses buffer secara bersamaan.
Jika buffer kosong maka consumer menunggu
Jika buffer penuh maka produser menunggu
5. Maka hasil outputnya
 - Maka yang di tampilkan adalah hasil acakan dari produser dan buffer dan menyesuaikan jumlah inter yang terdapat dalam data yang tersedia

PENUTUP

Kesimpulan

Dari hasil praktikum, dapat disimpulkan bahwa proses kompilasi dan eksekusi program C di Linux melalui beberapa tahap, yaitu *preprocessing*, *kompilasi*, *assembly*, dan *linking*. Setiap program yang dijalankan menunjukkan pentingnya penggunaan *thread* untuk menjalankan beberapa proses secara bersamaan agar lebih efisien dan teratur.

Saran

Untuk pengembangan lebih lanjut, Disarankan agar mahasiswa terus berlatih memahami penggunaan *thread*, *mutex*, dan *synchronization* agar dapat menghindari kesalahan saat program dijalankan. Selain itu, penulisan kode yang rapi dan diberi komentar akan membantu dalam memahami dan mengembangkan program di kemudian hari.